vLLM Inference on EKS — Architecture & Trade-offs

Production AI Model Serving Infrastructure

1. Design Decisions

EKS Node Group Configuration

Choice: g4dn.xlarge (1× NVIDIA T4 GPU, 16 GB VRAM, 4 vCPU, 16 GiB RAM) with Spot capacity and on-demand fallback.

The T4 GPU offers the best inference cost-to-performance ratio for models in the 100 MB–2 GB range. At $0.53/hr on-demand ($0.16/hr spot), g4dn is 3–5× cheaper than g5 or p3 instances. A single T4 comfortably hosts opt-125m (250 MB weights) with room for KV cache and concurrent requests. The larger g5 family (A10G, 24 GB) was deferred — its additional VRAM provides no benefit at this model size and costs 2× more per hour.

Trade-off — Spot vs On-Demand: Spot instances reduce GPU costs by ~70% but risk mid-inference termination. For regulated financial workloads, request truncation is unacceptable. We mitigate this with a spot-to-on-demand fallback in the EKS managed node group. A long-term improvement: Karpenter with graceful node draining via Kubernetes `preStop` hooks that flush in-flight requests before termination.

AMI: AL2_x86_64_GPU. AWS-optimized Amazon Linux 2 with pre-installed NVIDIA drivers (535.x), EKS bootstrap scripts, and nvidia-container-runtime. This avoids a custom AMI build pipeline — the GPU node joins the cluster and is ready to schedule pods within 2–3 minutes.

Load Balancer: AWS ALB Ingress Controller. Chosen over Nginx Ingress because it provides native AWS integration: ACM TLS certificates, WAF attachment, access logs to S3, and automatic target group registration. The trade-off is vendor lock-in, but for an AWS-only ecosystem constraint, ALB is the correct choice.

vLLM Serving Settings

Parameter	Value	Why
`tensor-parallel-size`	1	opt-125m fits on a single GPU. Tensor parallelism adds inter-GPU communication overhead with zero throughput benefit at this scale.
`gpu-memory-utilization`	0.85 (GPU) / 0.4 (CPU)	GPU: 85% leaves 2.4 GB headroom for CUDA context and KV cache. CPU: 40% reserves ~1.9 GB on an 8 GB machine — ample for the 250 MB model plus runtime overhead.
`max-model-len`	2048	Sufficient for most inference tasks. Increasing to 4096+ would require larger GPUs (A10G/A100) or quantization.
`max-num-seqs`	32 (GPU) / 16 (CPU)	Balances throughput against latency. At 32 concurrent sequences, each receives ~425 MB of KV cache budget. CPU is halved to reduce memory pressure on resource-constrained dev machines.
`dtype`	auto	vLLM auto-selects FP16 for GPU, FP32 for CPU. No manual override needed.

CPU-mode architecture: The Dockerfile uses a multi-stage build with two independent targets. The `cpu` target inherits from the official `vllm/vllm-openai-cpu` image (version pin: v0.22.1), which ships a CPU-optimized vLLM build with proper platform detection — the version string contains `+cpu` which triggers vLLM's CPU codepath. The `gpu` target uses `vllm/vllm-openai:v0.22.1-cu129`. This separation avoids the broken device detection that occurs when CUDA_HOME is set but no CUDA runtime exists — a known pitfall we encountered during development.

Kubernetes collision avoidance: The pod spec sets `enableServiceLinks: false` to prevent Kubernetes from injecting `VLLM_PORT=tcp://<service-ip>:8000` into the pod environment. vLLM's internal `envs.py` reads this variable directly for inter-process port allocation, and the URI format causes a `ValueError` crash in the engine core subprocess. The entrypoint script also unsets `VLLM_PORT` if it contains a `tcp://` prefix as a defense-in-depth measure.

Autoscaling Strategy

Choice: Kubernetes HPA (Horizontal Pod Autoscaler) with CPU and memory utilization targets.

HPA is native to EKS — no additional operator installation required. The metrics-server is pre-installed via the EKS module. For GPU workloads, CPU/memory metrics are secondary indicators — vLLM can be GPU-saturated at low CPU usage. The production upgrade path is:

1. Deploy NVIDIA DCGM Exporter as a DaemonSet (already configured via Helm values)
2. Configure Prometheus Adapter to expose `DCGM_FI_DEV_GPU_UTIL` as a custom metric
3. Add GPU utilization to the HPA spec with an 80% average target
4. Optionally adopt KEDA for request-queue-depth scaling using the Prometheus scaler

Scaling behavior: Scale-up is conservative — 1 pod every 60 seconds because GPU pods need 2–5 minutes for model loading on new nodes. Scale-down uses a 5-minute stabilization window to prevent thrashing from bursty inference traffic. The maximum replica count (5) is bounded by the GPU node group's maximum size, ensuring the cluster autoscaler has capacity headroom.

Why not KEDA initially? KEDA adds an operator dependency, custom resource definitions, and a metrics adapter. For an initial deployment targeting CPU/memory scaling, HPA is simpler, sufficient, and easier to debug. KEDA becomes valuable when we need event-driven scaling based on request queue depth — a clear upgrade path documented in the project README.

---

2. GPU Cluster Management at Scale

Fleet Management (10–50 Nodes)

Managing a GPU fleet at this scale demands treating GPUs as first-class, finite resources — their availability, health, and cost dominate operational concerns.

Multi-AZ Placement: GPU nodes are distributed across 2–3 availability zones within `me-central-1`. EKS managed node groups span all configured private subnets. This protects against single-AZ failures while keeping inter-AZ latency under 2ms. The trade-off: cross-AZ data transfer incurs cost ($0.01/GB), but this is negligible compared to GPU instance costs.

Node Lifecycle — Why Karpenter replaces managed node groups at scale: Static managed node groups work for 1–5 nodes but become rigid at 10+. Karpenter provides: - **Dynamic provisioning:** Selects the optimal GPU instance type based on pending pod requirements (GPU count, memory, architecture) - **Consolidation:** Identifies and replaces underutilized nodes, reducing idle GPU cost - **Spot interruption handling:** Receives the 2-minute rebalance recommendation and cordons the node before termination - **Drift detection:** Replaces nodes when the AMI or userdata changes

Spot Instance Strategy at Scale:

Diversify across multiple GPU instance types (`g4dn.xlarge` , `g5.xlarge` , `g4dn.2xlarge`) across multiple Spot capacity pools. Karpenter's `weighted` provisioning biases toward the cheapest available option. The fallback chain is: **Spot g4dn → On-Demand g4dn → On-Demand g5**. This ensures capacity availability even during Spot shortages.

Interruption handling: Kubernetes 1.30+ Pod Disruption Budgets with `unhealthyPodEvictionPolicy: AlwaysAllow`. Combined with vLLM's `preStop` hook, pods receive a 2-minute SIGTERM before node termination — sufficient to complete in-flight requests and flush pending generations to a backup endpoint.

GPU Health Monitoring — DCGM Deep Dive

NVIDIA's Data Center GPU Manager (DCGM) exposes 100+ GPU metrics. Four critical signals demand automated response:

Signal	Threshold	Action
XID Errors	XID 48 (Double-Bit ECC)	Immediate cordon + drain + terminate instance
Memory ECC Errors	Correctable errors > threshold in 5m window	Alert → cordon if sustained
GPU Temperature	>85°C sustained	Alert — thermal throttling degrades performance
Retired Pages	>10 pages	Drain node → terminate instance

Automated remediation pipeline: DCGM metric breaches → Prometheus alert fires → Alertmanager routes to webhook → webhook executes `kubectl cordon <node> && kubectl drain <node> --ignore-daemonsets --delete-emptydir-data` → Karpenter replaces the terminated instance. This entire flow completes in under 5 minutes without human intervention.

Driver cohesion: All nodes must run identical NVIDIA driver versions (pinned to CUDA 12.9). At scale, a custom AMI built with Packer — with the driver version hash baked into the AMI ID — prevents version drift. The EKS node group references this AMI, and Karpenter's drift detection triggers replacement when a new AMI is published.

---

3. What We Cut

The following items were deprioritized due to time constraints. They represent the implementation order for a production rollout.

Priority 1 — Must Implement Before Production

Item	Impact	Why Deprioritized
TLS/HTTPS at ALB	Exposes inference API over plain HTTP. ACM certificate + HTTPS listener + HTTP→HTTPS redirect is a one-line ALB annotation change.	Trivial to add; not needed for local dev validation.
AWS WAF	No OWASP protection, IP rate limiting, or bot control. Critical for a public-facing banking API.	Requires defining WAF rules and testing false-positive rates — deferred pending traffic pattern analysis.
GPU-Aware Autoscaling	Current HPA uses CPU/memory — suboptimal for GPU workloads. DCGM + Prometheus Adapter integration needed.	Requires Prometheus Adapter installation and metric mapping — documented as the immediate upgrade path.
External Secrets Operator	Kubernetes Secrets are manually created. ESO would sync HF_TOKEN from AWS Secrets Manager with automatic rotation.	Simple Helm install; deferred because opt-125m is a public model requiring no token.
ArgoCD GitOps	Current deployment uses `kubectl set image` via GitHub Actions. GitOps provides declarative state, drift detection, and immutable audit trail.	Requires ArgoCD installation + Application CRD setup — documented in CI/CD workflow comments.
Model Version Registry	No mechanism to track deployed model version or A/B test variants.	Requires MLflow or S3 versioning infrastructure — deferred for initial single-model deployment.

Priority 2 — Near-Term Enhancements

Istio Service Mesh: mTLS between control plane and inference pods, request-level telemetry (latency, error rate per model version), circuit breaking for degraded backends, and fault injection for resilience testing.

PagerDuty Integration: Alertmanager → PagerDuty for production incident response with on-call rotation and escalation policies.

GPU Auto-Remediation: Currently, GPU errors require manual `kubectl cordon/drain`. The DCGM→Prometheus→webhook pipeline described in Section 2 eliminates this manual step.

Shadow/Canary Deployments: Flagger or Argo Rollouts for progressive delivery of new model versions, with automated latency and error rate analysis before promoting to stable.

Cost Allocation: Granular AWS cost tags per model, team, and environment. GPU costs dominate at scale — without attribution, cost optimization is guesswork.

Priority 3 — Long-Term Platform Features

Multi-Model Serving: vLLM supports serving multiple LoRA adapters from a single base model. For a financial institution, this enables model-per-use-case (fraud detection, customer support, document analysis) on shared GPU infrastructure without provisioning separate clusters.

Model Quantization (AWQ/GPTQ): 4-bit weight quantization reduces opt-125m VRAM from ~250 MB to ~60 MB, enabling 4× higher concurrency per GPU or the use of smaller/cheaper GPU instances.

GPU Sharing via MIG: On A100/H100 instances, Multi-Instance GPU partitions a single physical GPU into isolated slices with dedicated memory and compute. This guarantees resource isolation for multi-tenant workloads — critical when different teams share the same GPU cluster.

Cross-Region Disaster Recovery: Active-passive EKS cluster replication with model artifact synchronization. Given UAE data residency requirements, a secondary Availability Zone within me-central-1 serves as DR, with the option to expand to a future secondary UAE region.

---

4. Data Residency & Compliance

This infrastructure is designed to operate under UAE Central Bank (CBUAE) regulatory requirements for financial data handling.

Data Residency (UAE Only)

Layer	Enforcement Mechanism
Region Lock	All Terraform resources deploy exclusively in `me-central-1`. An AWS Organizations SCP denies resource creation in any other region.
Model Storage	Model weights downloaded from HuggingFace Hub are cached on encrypted EBS volumes (gp3, AES-256) within UAE. A VPC endpoint for S3 reduces cross-border data transit if HuggingFace CDN serves from non-UAE edge locations.
Inference Data	Prompt and completion payloads never leave the VPC. vLLM runs entirely within private subnets. No external logging, analytics, or monitoring services receive inference payloads.
Log Data	CloudWatch Logs, VPC Flow Logs, and EKS audit logs remain in me-central-1. Retention: 30 days operational logs, 7 years for financial audit records in S3 Glacier Deep Archive.

Network Isolation

- **VPC Architecture:** GPU nodes in private subnets with no direct internet access. Outbound traffic (model download, ECR pull) routes through NAT Gateway in public subnets. Inbound traffic enters exclusively via ALB.
- **Security Groups:** GPU nodes allow ingress only from the VPC CIDR on port 8000 (vLLM API) and port 9400 (Prometheus metrics). The ALB security group is restricted to specific CIDR ranges in production.
- **VPC Endpoints:** ECR accessed via Interface VPC endpoint. Secrets Manager accessed via Interface VPC endpoint. No traffic traverses the public internet for AWS service calls.
- **Network Policies:** Calico or Cilium network policies restrict pod-to-pod communication — the vLLM pod accepts traffic only from the ALB controller and can initiate connections only to HuggingFace CDN ranges and the ECR VPC endpoint.

Audit Logging

Source	What is Logged	Retention
AWS CloudTrail	All AWS API calls (EKS, EC2, ECR, Secrets Manager, IAM). Organization trail with log file validation enabled.	90 days CloudWatch + 7 years S3
EKS Audit Logs	Kubernetes API server actions: who modified deployments, accessed secrets, ran `kubectl exec`.	30 days CloudWatch
VPC Flow Logs	Every IP flow 5-tuple (src/dst IP, port, protocol, accept/reject). Critical for data exfiltration detection.	30 days CloudWatch
ALB Access Logs	Request path, source IP, HTTP status, latency. Inference payloads are explicitly excluded to protect PII.	30 days S3

Secrets Management

- **HuggingFace Token (when needed):** Stored in AWS Secrets Manager under `mal-vllm-huggingface-token`, encrypted with a customer-managed KMS key, accessed via IRSA through the External Secrets Operator.
- **Model Integrity:** SHA256 checksums verified at model load time. Version pinning in deployment config prevents unauthorized model swaps.
- **Zero Secrets in Repository:** No API keys, tokens, or credentials appear in code. All sensitive values flow through Kubernetes Secrets injected from AWS Secrets Manager at runtime.

CBUAE-Specific Considerations

- **Data Localization:** Prompt data, generated completions, and model fine-tuning data remain in UAE at all times. The `me-central-1` region satisfies this requirement under current CBUAE guidelines.

- **Right to Audit:** CloudTrail + EKS audit logs provide an immutable, timestamped record of every administrative action for regulatory examination.
- **Data Classification:** All S3 buckets, EBS volumes, and CloudWatch log groups tagged with PIFI data classification levels. SSE-KMS encryption with customer-managed keys (not AWS-managed) enforced.
- **Key Management:** KMS keys are single-region (me-central-1), single-tenant (one key per service), with automatic annual rotation enabled.

---

5. 100x Scale — Nightmare Scenario

Scenario: Inference traffic surges 100× overnight — a viral marketing campaign or competitor outage directs unexpected load to the inference endpoint.

What Breaks First

Failure Point	Root Cause	Time to Failure
GPU Capacity	1× g4dn.xlarge handles ~8 req/s at 32 concurrent sequences. 100× = 800 req/s requiring ~50 GPU nodes. AWS GPU Spot capacity in me-central-1 is limited and unpredictable.	5–10 minutes (HPA maxes out, pods stuck Pending)
Model Cold Start	New GPU node: 2–3 min EKS join + 3–5 min driver init + 1–2 min model download. Total ~8 min/node cold start means capacity arrives minutes after the spike peaks.	15–20 minutes
ALB Connection Pool	ALB default idle timeout is 60 seconds. At 800 req/s, connection pools exhaust → TCP resets to clients.	10–15 minutes
Prometheus OOM	50 nodes × 100+ DCGM metrics at 15s scrape interval = 5,000+ active time series. Prometheus memory balloons.	30 minutes
ECR Pull Throttling	50 nodes pulling a 10+ GB GPU image simultaneously can trigger implicit ECR rate limits.	10 minutes
HuggingFace CDN	50 independent model downloads from HuggingFace Hub trigger rate limiting or bandwidth throttling.	15 minutes

Design Changes Required

1. **Warm GPU Pool:** Maintain 5–10 pre-initialized nodes with model cached and drivers loaded. Karpenter minimum node counts per AZ handle this declaratively.
2. **Model Caching Layer:** Download model weights once into an S3 bucket. Use an init container that copies from S3 to a shared EFS volume mounted across all inference nodes. Eliminates per-node HuggingFace downloads entirely.
3. **Multi-Cluster Architecture:** Distribute inference across 2–3 independent EKS clusters in me-central-1. Route53 latency-based routing or AWS Global Accelerator distributes requests. This also diversifies GPU capacity requests across multiple AWS placement groups.
4. **Request Queuing + Backpressure:** Deploy an Envoy sidecar proxy implementing token-bucket rate limiting with configurable burst capacity. Clients receive HTTP 429 with a `Retry-After` header instead of indefinite request queuing. The ingress controller is configured with surge queue depth limits.
5. **Model Quantization:** Switch from FP16 to AWQ 4-bit quantization, reducing VRAM per model from 250 MB to ~60 MB — enabling 4× concurrency per GPU without adding nodes.
6. **Ingress Evolution:** Replace ALB with NLB (supports millions of concurrent connections vs ALB's thousands) fronted by an Envoy proxy fleet handling connection pooling, retry budgets, health checking, and circuit breaking.
7. **Prometheus Federation:** Deploy lightweight Prometheus agents per cluster that forward to a centralized Thanos or Grafana Mimir instance for long-term storage and global querying. Avoids single-Prometheus memory limits.
8. **GPU Reservation System:** Implement Kubernetes priority classes and resource quotas per team. System-critical inference (fraud detection) receives higher scheduling priority than batch workloads (document analysis). Prevents noisy-neighbor problems during contention.

The Real Cost

At 100× scale, GPU infrastructure costs **$400–800/hour** (50 nodes × $8–16/hr on-demand). This forces: - **Reserved Instances** (1-year commitment) for baseline capacity (~60% savings over on-demand) - **Spot instances** for burst capacity above baseline - **Minimum 70% GPU utilization target** across the fleet — idle GPUs are a $ crisis - **Per-team chargeback** via Kubecost or CloudHealth to drive cost accountability

---

Document last updated: June 2026